BUS13-2

Designing Operations from Scratch

Systems thinking, Theory of Constraints, and flow analysis as a design lens

---

Can this actually be delivered? Where will it break first?

Albert School — 22 hours

Contents

1 Systems thinking as an operational lens

An operation is a **system**: a set of parts whose interactions produce behavior that none of the parts has on its own.

> **Definition 1** A **stock** is a quantity that accumulates over time (inventory, cash, orders waiting, work-in-progress). A **flow** is a rate that changes a stock (orders arriving per hour, units produced per day). Stocks are measured in units; flows in units per unit of time.

> **Definition 2** **Emergent behavior** is system behavior that arises from the interaction of parts and cannot be read off from any part in isolation.

> **Definition 3** **Suboptimization** is improving one part of a system in a way that degrades the performance of the system as a whole.

To see an operation as a system is to track its stocks and flows, and to ask how a local change propagates through the whole rather than judging each part on its own.

2 Constraint identification: the Theory of Constraints

> **Definition 4** The **constraint** (or bottleneck) of a system is the part that limits the throughput of the whole. Every system has at least one.

> **Definition 5** A **physical constraint** is a limit set by a tangible resource (a machine, a person, a supply). A **policy constraint** is a limit set by a rule, procedure, or decision. An **organizational constraint** is a limit set by how roles, responsibilities, or incentives are structured.

The **Theory of Constraints** holds that the performance of a system is governed by its constraint, so improvement effort must be directed there.

Definition 6 The **Five Focusing Steps**:
1. **Identify** the system's constraint.
2. **Exploit** the constraint — get the most from it without further investment.
3. **Subordinate** everything else to the decision made in step 2.
4. **Elevate** the constraint — increase its capacity.
5. If the constraint has been broken, **return to step 1**; do not let inertia become the constraint.

3 Throughput vs. utilization

Definition 7 **Throughput** is the rate at which a system produces completed output. **Utilization** is the fraction of available capacity that is in use,

$$\rho = \frac{\text{arrival rate}}{\text{service capacity}}.$$

A system can be highly utilized yet deliver poor throughput, because high utilization produces long queues.

Proposition 8 As utilization ρ approaches 1, waiting time grows without bound. For a single-server queue with variability, average waiting time is proportional to

$$\frac{\rho}{1-\rho}.$$

The **utilization–delay curve** is the graph of this relationship: nearly flat at low utilization, then rising sharply as $\rho \to 1$. Beyond roughly $\rho = 0.85$, small increases in utilization produce large increases in delay.

Definition 9 **Slack** is unused capacity held deliberately. It is **strategic capacity**: the margin that keeps delay bounded and absorbs variability.

4 Queues, delays, and feedback loops

Theorem 10 (Little's Law) For a stable system, the average number of items in the system equals the average arrival rate times the average time an item spends in the system:

$$L = \lambda W,$$

where L is work-in-progress (items in system), λ is throughput (arrival rate of completed flow), and W is the average lead time.

Little's Law lets any one of the three quantities be estimated from the other two; e.g. lead time $W = \frac{L}{\lambda}$.

Definition 11 A **visible queue** is work waiting that can be seen and counted directly. An **invisible queue** is work waiting that is not directly observable (queued digital requests, pending decisions, deferred tasks).

Definition 12 A **feedback loop** exists when an output of the system feeds back to affect its own input. **Batch size** is the number of items processed together before the batch moves on; larger batches raise work-in-progress and lengthen delays.

5 Process design from scratch

Designing a process means specifying how value moves from customer request to delivery.

Definition 13 A **value delivery flow** is the sequence of steps that transforms a customer order into a delivered outcome. A **decision point** is a step where the path branches according to a condition. A **handoff** is a transfer of work from one party to the next.

To design a process from scratch: map the value delivery flow end to end, mark each decision point and each handoff, and design so that the path of any order is clear.

6 Make vs. buy decisions

Definition 14 A **make-vs-buy decision** chooses between building a capability internally (**make**) and obtaining it from an outside provider (**buy**).

Evaluate by comparing:

- **internal capability cost**: the cost to build and operate the capability, including **coordination overhead** — the cost of managing the work across the organization;
- **outsourcing cost**: the price paid to the provider, plus **quality risk** and **dependency** on that provider.

Choose the option with the lower total cost once coordination overhead and risk are included.

7 Unit economics as operational constraints

Definition 15 **Unit economics** describe the revenue and cost of a single unit of output: **margin per unit** (price minus variable cost), the split between **variable** and **fixed** costs, and the **break-even volume**

$$V_{\text{break-even}} = \frac{\text{fixed cost}}{\text{margin per unit}}.$$

Unit economics bound process design: the margin per unit caps what can be spent per unit on the process, the cost structure sets which volumes are viable, and the break-even volume fixes the throughput the operation must sustain. A process design that violates these bounds is not feasible regardless of its other merits.

8 Service design

Definition 16 A **service delivery experience** is the sequence of interactions through which a customer receives a service, physical or digital. A **touchpoint** is a point of contact between customer and service. A **failure point** is a touchpoint where the service can fail to deliver. A **recovery path** is the designed response that restores the service after a failure.

To design a service: map the touchpoints in order, identify at each one how it can fail, and specify a recovery path for each failure point.

9 Operational MVP

Definition 17 An **operational MVP** (minimum viable operations) is the smallest operational design that can deliver and validate the value proposition. Designing one means deciding, for each part of the operation, whether to **build** it, **simulate** it, or **defer** it.

Scope an operational MVP by building only what is needed to deliver and learn, simulating what can be faked without building, and deferring what is not yet required — and justify each choice by what it lets the operation validate now.

10 Early failure mode analysis

Definition 18 A **failure mode analysis** of an operational design identifies where the design will break first: the **failure points**, the **volume** at which each is triggered, and a **mitigation** for each.

For an early-stage design, identify the first three failure points, estimate the volume at which each triggers, and propose a mitigation for each.

11 Second-order effects

Operational changes have **second-order effects**: consequences that follow from the system's reaction to the change rather than from the change directly.

Proposition 19 Relieving a constraint does not remove the constraint from the system; it **moves** it. Fixing one bottleneck reveals the next, because throughput is then limited by whatever is now the binding constraint.

This is why local efficiency gains can degrade system performance: making a non-constraint faster adds work-in-progress ahead of the constraint without raising throughput, lengthening delays. Predicting the second-order effect of an improvement means asking what the system's constraint becomes once the change takes effect.

12 Presenting an operational design

An operational design is presented to a non-operations audience — investors, co-founders, or strategy leads — by translating system dynamics into business consequences: what the design delivers, where it will break first, at what volume, and what that means for the business.